
SOP: Branded Invoice + SOW + Payment System

Standard Operating Procedure for Next.js Agency Sites
Version 1.0 | June 2026 | XeedlyAI

A single admin tool that lets you select a product tier, build a scope of work, generate a branded PDF invoice, and send it to a client via email with an embedded Stripe payment link. Payment constitutes acceptance of the SOW — no separate contract signing, no chasing signatures. One document, one payment, project starts.

The invoice IS the contract. The terms section states: "Payment constitutes acceptance of this scope of work." When the client pays, they've agreed to the scope.

System Architecture

```
Admin Portal (/admin/close)
|
v
5-Step Invoice Wizard
|
v
POST /api/admin/create-invoice
|
+--> Stripe: Create + finalize invoice (card payments)
+--> PDF: Generate branded invoice with @react-pdf/renderer
+--> Supabase Storage: Upload PDF (public URL)
+--> Supabase DB: Save invoice record
+--> Resend: Send branded email with PDF attached
|
v
Client receives email -> Opens PDF -> Clicks Pay -> Stripe hosted page
```

Prerequisites

Accounts Needed

- **Stripe** (live mode) — payment processing
- **Supabase** — database + file storage
- **Resend** — transactional email with custom domain
- **Vercel** — hosting + serverless functions
- **Custom domain** with DNS access (for email sender verification)

Tech Stack

- Next.js 15+ (App Router)
- TypeScript + Tailwind CSS
- @react-pdf/renderer (PDF generation on serverless)
- Stripe Node SDK
- Supabase JS client

PHASE 1: Foundation

Step 1 — Install Dependencies

```
npm install @react-pdf/renderer stripe @supabase/supabase-js
```

Note: @react-pdf/renderer v4.x is required for React 19 compatibility.

Step 2 — Database Setup (Supabase)

Run this SQL in your Supabase SQL Editor. Creates three tables (customers, deals, invoices) plus a public storage bucket for PDFs.

```
CREATE TABLE IF NOT EXISTS customers ( ... );
CREATE TABLE IF NOT EXISTS deals ( ... );
CREATE TABLE IF NOT EXISTS invoices (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  invoice_number TEXT UNIQUE NOT NULL,
  deal_id UUID REFERENCES deals(id),
  customer_id UUID REFERENCES customers(id),
  status TEXT DEFAULT 'sent',
  line_items JSONB NOT NULL,
  build_total INTEGER NOT NULL,
  service_monthly INTEGER NOT NULL,
  stripe_invoice_id TEXT,
  stripe_invoice_url TEXT,
  pdf_url TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Storage bucket for PDFs
INSERT INTO storage.buckets (id, name, public)
VALUES ('invoices', 'invoices', true);
```

Full SQL including all columns is in `supabase/migrations/invoice_system.sql`

Step 3 — Environment Variables

Set these in your Vercel project settings (NOT in .env files that get committed):

Variable	Where to get it
STRIPE_SECRET_KEY	Stripe > Developers > API keys (sk_live_)
STRIPE_PUBLISHABLE_KEY	Same page (pk_live_)
STRIPE_WEBHOOK_SECRET	Stripe > Webhooks > endpoint > Signing secret
RESEND_API_KEY	Resend dashboard > API keys
RESEND_FROM_EMAIL	e.g., billing@yourdomain.com
NEXT_PUBLIC_SUPABASE_URL	Supabase project settings
SUPABASE_SERVICE_ROLE_KEY	Supabase > Settings > API
VENMO_HANDLE	Your Venmo username (e.g., @xeedly)

VENMO_QR_URL	URL to Venmo QR code image (optional)
NEXT_PUBLIC_SITE_URL	Your deployed URL

CRITICAL: Never commit real API keys to git. Use .env.local (gitignored) locally and Vercel env vars for production.

PHASE 2: Stripe Configuration

Step 4 — Stripe Dashboard Settings

- **Payment terms:** Settings > Billing > Invoices > set "Due upon receipt" — makes Stripe show "Due Today"
- **Payment methods:** Settings > Payment methods > enable Card. ACH is optional (2-week clearing).
- **Webhook:** Developers > Webhooks > Add endpoint > URL: `https://yourdomain.com/api/webhooks/stripe` > Events: `invoice.paid`, `payment_intent.succeeded`, `checkout.session.completed`
- **Branding:** Settings > Branding > upload logo and set colors for the hosted payment page

Step 5 — Resend Domain Verification

- Go to Resend > Domains > Add domain
- Add DNS records Resend provides (SPF, DKIM, DMARC)
- Verify the domain — now you can send FROM `billing@yourdomain.com`
- Set REPLY-TO to your personal email so client replies go to your inbox

PHASE 3: Code Implementation

Step 6 — Stripe Helper Functions

Add to `src/lib/stripe.ts`: `getOrCreateStripeCustomer()` and `createStripeInvoiceWithSOW()`. Key settings:

```
collection_method: "send_invoice" // Don't auto-charge
days_until_due: 0 // Due upon receipt
payment_method_types: ["card"] // Card only
```

Finalize the invoice but do NOT send it via Stripe — we send our own branded email.

Step 7 — PDF Invoice Generator

Create `src/lib/invoice-pdf.tsx` using `@react-pdf/renderer`. Use fontsource CDN for fonts (NOT Google Fonts — variable TTFs crash fontkit).

PDF sections: Header with logo > Invoice metadata > Billing block > Comparable value banner > Build deliverables table > Total due now (build only) > Monthly service note > Service deliverables checklist > Payment options > Terms > Partnership statement > Footer

Key rule: TOTAL only includes build items. Service items are shown as included scope.

Step 8 — Email Template

Create `src/lib/email-templates/invoice-email.ts`. Branded HTML with logo, "Pay Invoice Online" CTA button, "Pay with Venmo" secondary CTA, PDF attached via Resend API.

FROM: `billing@yourdomain.com` | REPLY-TO: your personal email (threads to your inbox)

Step 9 — Invoice API Route

Create `/api/admin/create-invoice/route.ts` — the orchestrator:

1. Generate invoice number (INV-YYYY-NNN, sequential)
2. Upsert customer in Supabase + Stripe
3. Create deal record in Supabase
4. Create Stripe invoice (build items only, finalized not sent)
5. Generate branded PDF with Stripe payment URL embedded
6. Upload PDF to Supabase Storage (public bucket)
7. Save invoice record to database
8. Send branded email with PDF attached via Resend

Step 10 — Admin Close Flow UI

Build a 5-step wizard at /admin/close:

1. Build Tier Selection + "Full Service Included" toggle
2. Service Tier Selection (Foundation / Growth / Authority monthly pricing)
3. Customer Info form (name, email, phone, company, notes)
4. SOW Builder (editable line items, terms, comparable value statement)
5. Invoice Preview + "Send Invoice" button

PHASE 4: Go Live

Step 11 — Testing

- Send a test invoice to yourself
- Verify PDF renders correctly (logo, fonts, layout, no \$0.00 on included items)
- Click Stripe payment link — confirm live mode (no "Test Mode" banner)
- Check Venmo link opens correctly
- Verify email has PDF attached and reply-to threads correctly
- Void test invoices in Stripe dashboard after testing

Step 12 — Stripe Live Mode Verification

- Invoice URL should contain /live_ not /test_
- No "TEST MODE" badge on the payment page
- If you see test mode: check STRIPE_SECRET_KEY in Vercel starts with sk_live_
- Redeploy after changing any env vars

Daily Operations

Sending an Invoice

1. Go to /admin/close
2. Select build tier + toggle full service
3. Select service tier
4. Enter customer info
5. Review/edit the SOW
6. Preview the invoice
7. Click "Send Invoice"
8. Client receives branded email with PDF + payment link

When Client Pays

- Stripe webhook fires invoice.paid
- Deal status updates automatically
- Build timer starts
- Monthly service begins 30 days from payment date

Troubleshooting

Problem	Solution
"Test Mode" on Stripe page	STRIPE_SECRET_KEY is sk_test_. Replace with sk_live_ and redeploy
"Could not resolve font"	Don't use fontStyle italic without registering italic font file
"Offset outside DataView"	Font URL serves variable TTF. Switch to fontsource CDN
Font 404 errors	Google Fonts changes URLs. Use fontsource CDN instead
Email not sending	Check RESEND_API_KEY is set. Verify domain in Resend
PDF not uploading	Create Supabase Storage bucket 'invoices' with public access
"Due tomorrow" not "Due today"	Stripe > Billing > Invoices > Payment terms > Due upon receipt
Bank details on invoice	Never put YOUR bank info on invoices. Stripe handles ACH securely
Live keys in git	Roll the key in Stripe immediately. Use .env.local (gitignored) only

Security Checklist

- .env.local is in .gitignore
- .env.local.example contains only placeholder values, never real keys
- Stripe key in Vercel starts with sk_live_
- No bank account numbers appear anywhere in invoice PDFs or emails
- Admin routes are protected by auth middleware
- Webhook endpoint validates Stripe signature
- Supabase uses service role key (server-side only, never exposed to client)
- REPLY-TO is set so client replies go to your inbox

Cost of Operation

Service	Cost
Stripe	2.9% + 30c per card transaction
Resend	Free up to 3,000 emails/month, then \$20/mo
Supabase	Free tier covers most agencies, Pro at \$25/mo

Vercel	Free tier for hobby, Pro at \$20/mo
@react-pdf/renderer	Free (open source)

Total fixed cost to operate: \$0-65/month depending on tier selections.

Built by XeedlyAI. This system is production-tested and powers real client invoicing at xeedly.com.